

Tailor Moments — Executive Summary

ARCHITECTURE REVIEW & ROADMAP · 30 JUNE 2026 · FULL DETAIL
IN `DOCS/PRODUCT-ROADMAP-2026.MD`

The one thing that matters this week

Production email is not configured (`TM_ACS_*` empty). **Bookings never reach wineries.** A customer can complete a booking today and the winery approval link is only written to server logs — never sent. Password resets don't send either. Until this is fixed, nothing else on the roadmap matters. **Fix: configure ACS or swap to Resend (~an afternoon).**

Three more fixes before any new features (P0, 1–2 weeks, all backend/ops)

1. **Enforce Turnstile** — the secret is unset; bots can create real bookings.
2. **Staging environment** — every bug this month was found by us, in production.
3. **CI + auto-deploy** — deploys are manual wrangler runs from a laptop; we repeatedly lost hours to “is my fix actually live?”

The verdict on the architecture

Keep it. The stack is lean, cheap (<\$150/mo) and correctly shaped: static frontend on Cloudflare, Functions + Postgres API, repository pattern, action-token approvals, logged itinerary engine. No rewrite needed. Two structural debts to pay before they get expensive:

- **Partner model is hard-coded** (role enum + `transport_company` string on the user). Generalize to a `partner` entity *before* adding partner type #3 (restaurants, hotels, shipping). Cheap now, surgery later.
- **No shared API contract** — frontend and backend hand-duplicate types; we already built the same homepage twice. One OpenAPI spec fixes drift *and* becomes the B2B API + the AI-platform integration surface. One artifact, three payoffs.

The commercial model, translated to build

Decision (Anh)	What it means technically
% rev-share on bookings	Stripe Connect destination charges — customer pays us, our fee auto-retained, partner paid out by Stripe. Zero invoicing ops for a 2-person team.
Partner SaaS tier later	Start logging booking attribution + revenue events now so the analytics tier has history at launch.
Connect to partner booking systems	Rezdy discovery spike (2–3 days) : their agent model may hand us availability + booking + commission in one integration. Fallback for everyone else: .ics calendar invite on confirmation emails — works with Outlook & Google, no OAuth, ~1 day of work.
Wine shipping to Asia	Partner it, never build it (export compliance). v1 = “ship my wine home” button → ops queue → vetted partner + referral fee.

Roadmap at a glance

- **P0 (now)**: email live · Turnstile · staging · CI. *Makes the existing loop real.*
- **P1 (1–2 mo)**: Stripe deposits · pricing in DB · customer manage/cancel · partner-entity refactor · Rezdy spike. *Puts money in the loop.*
- **P2 (2–4 mo)**: Connect payouts (the rev-share engine) · OpenAPI contract · availability sync · partner analytics (free for founding partners → paid tier).
- **P3 (3–6 mo)**: **One MCP server** that serves Claude/ChatGPT/Gemini booking agents, the WhatsApp/WeChat concierge, *and* our internal AI ops agents · post-trip automation · wine-shipping v1.

Cost posture

Cloud spend is not the problem — founder time is. Biggest saving available: the winery↔winery travel matrix is fixed (~40 venues) — precompute it and Redis + most of OSRM can be retired. Do it opportunistically, not urgently.

Split of work

- **Sean**: everything in P0 (config + infra), then Stripe + partner entity + OpenAPI.
- **Anh**: .ics invites (once email lives), customer booking-manage UI, founding-partner onboarding flow. Frontend Estate redesign is 100% complete and live as of today.

Bottom line: the product is architecturally sound and now looks the part end-to-end. It cannot make money until email works and payments exist. Those two things, in that order, are the company for the next eight weeks.